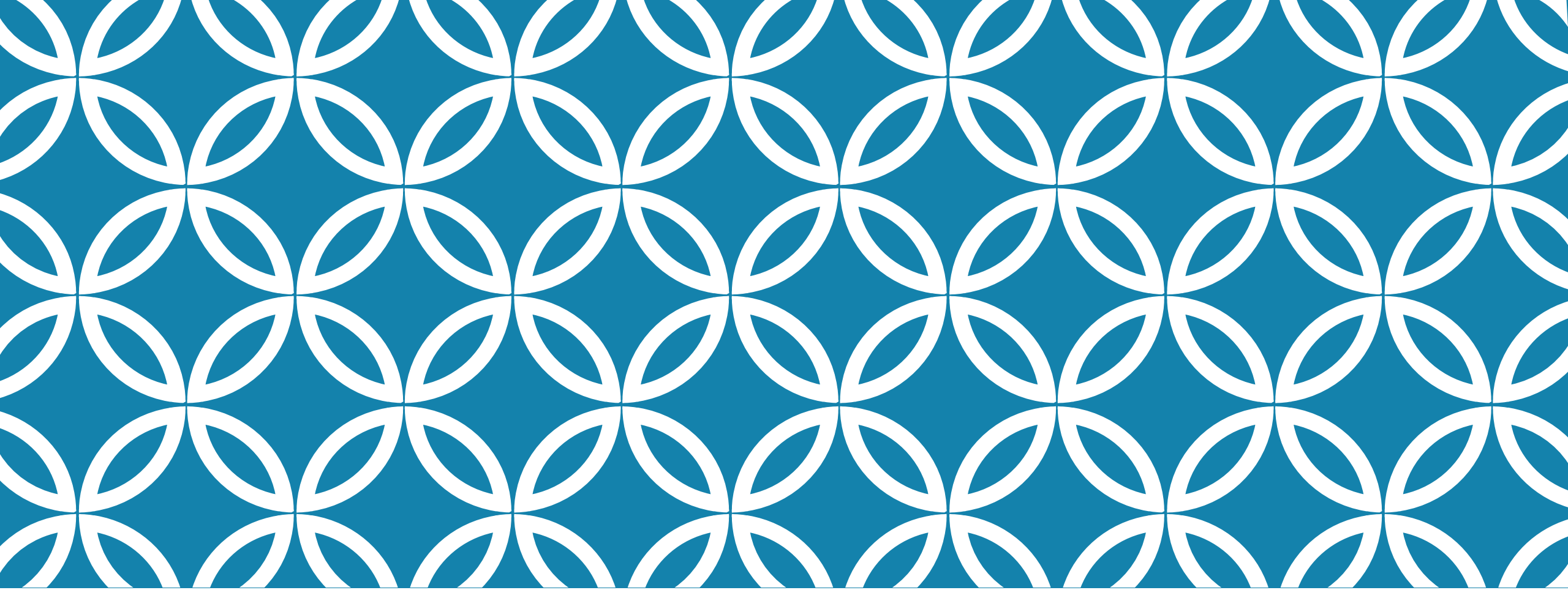


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

ÇOKLU MİRAS ALMAYA GİRİŞ

- Daha önce tekli miras alma kavramını öğrenmiştik.
- Tekli mirasta bir sınıf başka bir sınıfın özelliklerini devralıyordu.
- Peki ya bir sınıf birden fazla sınıftan özellik devralırsa ne olur?
- İşte bu duruma çoklu miras alma (multiple inheritance) denir.
- Gerçek hayattaki en basit örnek: bir çocuk hem annesinden hem babasından özellikler alır.
- Python programlama dilinde bu mantığı rahatlıkla uygulayabiliriz.

ÇOKLU MİRAS ALMA MANTIĞI

- Çoklu mirasta bir sınıf birden fazla üst sınıftan kalıtım alır.
- Bu sayede farklı sınıfların işlevlerini tek bir sınıfta birleştirmiş oluruz.
- Python bu özelliği esnek bir şekilde destekler.
- Ancak bu esneklik bazı karmaşık durumları da beraberinde getirir.
- Özellikle aynı isimde fonksiyon veya özellik olduğunda dikkatli olunmalıdır.
- Bir örnekle daha net hale getirelim.

TEMEL KOD ÖRNEĞİ

- Burada Cocuk sınıfı hem Anne hem de Baba sınıfından miras alır.
- Yani tek bir sınıf içinde iki ayrı sınıfın özelliklerini kullanabilir.
- Çocuğun hem anneden hem babadan nitelik alması mantığı kodda birebir gösterilmiş olur.
- Çalıştırdığımızda her iki sınıfın fonksiyonlarına da erişebiliriz.
- Bu, çoklu mirasın temel işleyişidir.

```
class Anne:
    def print_Anne(self):
        print("Anne sınıfının özellikleri")

class Baba:
    def print_Baba(self):
        print("Baba sınıfının özellikleri")

class Cocuk(Anne, Baba):
    pass
```

KODUN ÇIKTISI

- Görüldüğü üzere Cocuk sınıfı hem print_Anne() hem de print_Baba() fonksiyonlarını çağırabildi.
- Bu, birden fazla sınıfın özelliklerinin tek bir yerde birleştiğini kanıtlıyor.
- Çoklu miras bu basit örnekle anlaşılabilir.
- Ancak işler aynı isimli fonksiyonlar olduğunda biraz karışır.

```
object_d = Cocuk()
object_d.print_Anne()
object_d.print_Baba()
```

Çıktı:

```
Anne sınıfının özellikleri
Baba sınıfının özellikleri
```

AYNI İSİMDE FONKSİYON SORUNU

- Hem Anne hem de Baba sınıfında aynı isimli bir fonksiyon var: **goz_rengi( )**.
- Peki, **Cocuk** sınıfı bu fonksiyonu hangi sınıftan miras alacak?
- Python bu durumda bir öncelik kuralı uygular.
- İşte çoklu mirasın en kritik noktalarından biri budur.

```
class Anne:
    def goz_rengi(self):
        print("Anne sınıfının özellikleri")

class Baba:
    def goz_rengi(self):
        print("Baba sınıfının özellikleri")
```

ÖNCELİK SIRASI

- Python öncelik olarak ilk yazılan sınıfı dikkate alır.
- Burada Cocuk sınıfı önce Anne'den sonra Baba'dan miras aldı.
- Bu nedenle goz_rengi( ) fonksiyonu Anne'den geldi.
- Eğer sıralamayı değiştirirsek sonuç da değişir.

```
class Cocuk(Anne, Baba):
    pass

object_d = Cocuk()
object_d.goz_rengi()
```

Çıktı:

```
Anne sınıfının özellikleri
```

BABA ÖNCELİKLİ MİRAS

- Şimdi öncelik Baba sınıfına geçti.
- Çünkü **Cocuk** sınıfı önce Baba'yı yazdı.
- Bu durum projelerde kafa karışıklığına sebep olabilir.
- Büyük ekip çalışmalarında aynı isimli fonksiyonlarla çakışma riski artar.
- Bu yüzden çoklu miras kullanılırken planlama yapılmalıdır.

```
class Cocuk(Baba, Anne):
    pass

object_d = Cocuk()
object_d.goz_rengi()
```

Çıktı:

```
Baba sınıfının özellikleri
```

YAPICI FONKSİYONLAR (CONSTRUCTOR)

- Hem Anne hem de Baba sınıfında **__init__( )** fonksiyonu tanımlı.
- Cocuk sınıfını oluşturduğumuzda hangi sınıfın yapıcısı çalışır?
- Python yine öncelik sırasına bakar.
- İlk yazılan sınıfın yapıcı fonksiyonu çalışır.
- Bu örnekte Baba sınıfı önce yazıldığı için siyah göz rengi alınır.

```
class Anne:
    def __init__(self):
        self.goz_rengi = "Yeşil"
        print(self.goz_rengi)

class Baba:
    def __init__(self):
        self.goz_rengi = "Siyah"
        print(self.goz_rengi)

class Cocuk(Baba, Anne):
    pass
```

ÇIKTI VE YORUM

- Çocuğun göz rengi Baba'dan geldi.
- Çünkü Cocuk sınıfı önce Baba sınıfını miras aldı.
- Eğer Anne önce yazılsaydı göz rengi Yeşil olurdu.
- Bu bize miras alma sırasının sonuç üzerinde doğrudan etkili olduğunu gösterir.
- Şimdi bu sıralamayı elle nasıl değiştirebiliriz ona bakalım.

```
object_d = Cocuk()
```

Çıktı:

```
Siyah
```

BELİRLİ BİR YAPICIYI ÇAĞIRMA

- Burada biz özellikle Anne sınıfının **__init__( )** fonksiyonunu çağırdık.
- Böylece öncelik kuralını elle değiştirmiş olduk.
- Python bize bu esnekliği verir.
- Ancak dikkat: sadece belirli fonksiyonları çağırırsınız, tüm sınıfı değil.
- Bu yöntem çoklu mirasta kontrolü geliştiriciye bırakır.

```
class Cocuk(Baba, Anne):
    def __init__(self):
        Anne.__init__(self)
```

Çıktı:

```
Yeşil
```

KARIŞIK ÖRNEK – HEM ANNE HEM BABA

- Bu örnekte çocuğun **göz rengi** Anne'den, **boyu** ise Baba'dan gelir.
- Çünkü biz **__init__( )**'i Anne'den çağırdık ama **boy( )** için sıralama Baba'yı öne koydu.
- Yani farklı özelliklerde farklı sınıflar öncelikli olabilir.
- Çoklu mirasın en güzel yanlarından biri budur.
- Hem seçicilik hem de esneklik sağlar.

```
class Anne:
    def __init__(self):
        self.goz_rengi = "Yeşil"
        print(self.goz_rengi)
    def boyu(self):
        print(1.70)

class Baba:
    def __init__(self):
        self.goz_rengi = "Siyah"
        print(self.goz_rengi)
    def boyu(self):
        print(1.86)

class Cocuk(Baba, Anne):
    def __init__(self):
        Anne.__init__(self)
```

ÇIKTI ANALİZİ

- Çocuk yeşil gözlü ve 1.86 boyunda oldu.
- Bu örnek bize sınıfların fonksiyon bazında ayrı ayrı seçilebileceğini gösteriyor.
- Yani sadece tek bir sınıfın tüm özelliklerini almak zorunda değiliz.
- Bu, projelerde özelleştirme yapmamızı kolaylaştırır.
- Ancak hangi fonksiyonun hangi sınıftan çağrıldığını takip etmek önemlidir.

```
Yeşil
1.86
```

TAMAMEN ANNE'DEN ÖNCELİK ALMA

- Gördüğünüz gibi hem göz renginde hem de boy özelliklerinde Anne'den miras aldık.
- Baba sınıfı ilk sırada yazılmış olsa bile, biz kod içinde Anne'i seçtik.
- Bu bize "öncelik sırasını değiştirme" özgürlüğü tanır.
- Çoklu mirasın gücü burada ortaya çıkar.
- Doğru kullanıldığında büyük esneklik sağlar.

```
class Cocuk(Baba, Anne):
    def __init__(self):
        Anne.__init__(self)
    def boy(self):
        return Anne.boy(self)
```

Çıktı:

```
Yeşil
1.75
```

MİRASIN ZİNCİRLEME ETKİSİ

- Burada **Cocuk** sınıfı başka bir sınıf olan **Torun**'a miras veriyor.
- Yani miras alınmış bir sınıf başka bir sınıfa tekrar miras bırakabilir.
- Buna **zincirleme miras** denir.
- Böylece bir özellik nesilden nesile aktarılır.
- Çoklu mirasın gücünü bu tür zincirlerde daha iyi görürüz.

```
class Torun(Cocuk):
    pass

torun = Torun()
torun.goz_rengi
torun.boy()
```

Çıktı:

```
Yeşil
1.75
```

GENEL DEĞERLENDİRME

- Çoklu miras alma Python’da güçlü bir özelliktir.
- Birden fazla sınıfın özelliklerini tek bir sınıfta birleştirmeyi sağlar.
- Ancak aynı isimli fonksiyonlarda çakışma olabilir.
- Öncelik sırası yazım sırasına göre belirlenir.
- Geliştirici gerekirse manuel olarak istediği sınıfın fonksiyonunu seçebilir.
- Zincirleme miras ile bu özellikler daha da genişletilebilir.